

Nix 配置架构与使用指南

AI Assistant (融合分析与实践)

2025 年 4 月 13 日

摘要

本文档旨在提供对当前 Nix 配置仓库的全面分析与实用指导。它首先阐述了配置核心理念、架构目标和所依赖的关键技术 (Nix Flakes)。接着,详细解析了代码库的模块化结构和设计原则。在此基础上,文档分步指导了如何在不同平台 (NixOS VM, macOS, WSL) 上进行首次环境部署。随后,介绍了用户环境管理 (Home Manager) 的实践方法和常见定制任务 (如包管理、环境修改、Overlays 使用)。最后,汇总了日常维护的工作流程和命令。本文的目标是结合架构理解与动手实践,帮助用户高效地使用、维护和扩展这套配置。

目录

1	概述与核心理念	3
1.1	架构目标	3
1.2	核心技术栈: Nix Flakes	3
1.3	flake.nix 的中心协调作用与 Makefile	3
2	架构设计与配置结构	4
2.1	模块化与抽象 (lib, modules)	4
2.2	关注点分离 (目录结构)	4
2.3	显式依赖与可复现性	5
3	系统配置管理与部署	5
3.1	跨平台支持	5
3.2	主机特化 (machines/)	5
3.3	首次环境部署实践	6

3.3.1	部署到 NixOS 虚拟机 (以 VMware Fusion 为例)	6
3.3.2	部署到 macOS (使用 nix-darwin)	7
3.3.3	构建并部署到 WSL	8
4	用户环境管理 (Home Manager)	9
4.1	用户配置结构 (users/)	9
4.2	常见任务: 管理软件包	9
4.3	常见任务: 修改用户环境	9
5	高级功能与定制	10
5.1	常见任务: 使用 Overlays 修改或添加包	10
5.2	常见任务: 添加新的系统配置	11
6	日常维护与工作流	12
6.1	应用配置更改	12
6.2	测试配置更改	12
6.3	更新 Flake 依赖	13
6.4	编译本文档	13
7	结论	13

1 概述与核心理念

1.1 架构目标

本 Nix 配置仓库旨在实现一套声明式、可复现、高度模块化的配置管理方案，其核心目标包括：

- **声明式与可复现**: 通过 Nix 的纯函数特性，确保任何时候的构建结果都是一致的，便于版本控制、回滚和跨设备部署。
- **模块化与可扩展**: 将复杂的系统和用户配置分解为可重用的、逻辑独立的模块，降低维护成本，提高扩展性。
- **跨平台一致性**: 利用统一的 Flake 接口管理 NixOS、macOS (nix-darwin) 和 WSL 环境，尽可能共享配置逻辑。
- **简化工作流**: 通过 Makefile 封装常用 Nix 命令，提供便捷的操作入口，降低日常使用门槛。

1.2 核心技术栈：Nix Flakes

Nix Flakes 是本配置的基石，提供了现代化的 Nix 包管理和项目结构规范：

- **显式依赖管理 ('inputs')**: 在 `flake.nix` 中明确声明所有外部依赖（如 `nixpkgs`, `home-manager`, `nix-darwin` 等），并通过 `flake.lock` 文件锁定其精确版本，保证了构建的确定性。
- **标准化的输出 ('outputs')**: `flake.nix` 定义了一组标准的输出类型，如 `nixosConfigurations`, `darwinConfigurations`, `homeConfigurations`, `overlays`, `pkgs`, `devShells` 等，使得配置的结构清晰，易于被其他 Flake 或工具消费。

1.3 `flake.nix` 的中心协调作用与 Makefile

- **`flake.nix`**: 作为配置仓库的**唯一入口点**，它 `orchestrates` 了所有内部模块和外部依赖，定义了最终可构建的系统和用户配置蓝图。它通常会调用内部库函数（如下文的 `lib/mkssystem.nix`）来组装最终的配置。
- **Makefile**: 提供了一层用户友好的抽象，封装了常见的 Nix Flake 命令。用户可以通过简单的 `make` 命令执行诸如系统切换 (`make switch`)、虚拟机引导 (`make`

vm/bootstrap)、测试构建 (make test) 等操作, 而无需记忆复杂的 nix build 或 nixos-rebuild 参数。

2 架构设计与配置结构

本配置采用模块化的设计思想, 将不同关注点的配置分离到独立的目录和文件中, 以提高可维护性和复用性。

2.1 模块化与抽象 (lib, modules)

- lib/: 存放可重用的 Nix 函数和逻辑。例如, lib/mkssystem.nix 可能是一个核心辅助函数, 负责接收参数 (如主机名、用户名、系统类型), 并根据这些参数加载、组合正确的机器配置、用户配置、模块和 Overlays, 最终生成一个完整的系统配置对象。这体现了配置构建逻辑的抽象。
- modules/: 存放跨机器、跨用户共享的 NixOS 或 Darwin 配置模块。这些模块封装了特定的功能 (如通用开发环境设置、特定服务配置), 可以在机器或用户配置中按需导入, 遵循 DRY (Don't Repeat Yourself) 原则。

2.2 关注点分离 (目录结构)

配置库的目录结构清晰地反映了关注点的分离:

- flake.nix: 顶层入口和依赖定义。
- machines/: **机器特定配置**。存放与特定硬件或虚拟机环境相关的系统级设置。每台主机 (物理机或 VM) 对应一个文件, 例如 machines/vm-aarch64.nix 可能包含 VMware 驱动或特定内核参数, 而 machines/macbook-pro-m1.nix 可能包含 Darwin 特有的设置。这些文件通常会导入 modules/ 中的通用模块。
- users/: **用户配置**。每个用户一个子目录。
 - <user>/(nixos|darwin).nix: 定义用户的系统级属性, 如 UID, GID, 所属组, 以及可能需要的系统级服务或全局包 (虽然包管理更推荐在 Home Manager 中进行)。
 - <user>/home-manager.nix: **核心用户环境配置**。利用 Home Manager 声明式地管理用户的 dotfiles (如 .zshrc, .gitconfig), 用户级服务, 用户安装的软件

包 (`home.packages`), 环境变量, GUI 应用设置等。这是个性化用户环境的主要场所。

- **overlays/:** **Nixpkgs 覆盖与扩展**。存放用于修改 `nixpkgs` 中现有包 (如打补丁、修改编译选项) 或添加自定义包定义的 Overlay 文件。这些 Overlays 通常由 `lib/mksystem.nix` 应用到最终的 `pkgs` 集合中。
- **pkgs/:** **自定义 Nix 包**。存放不适合放在 Overlay 中或更复杂的自定义软件包的 Nix derivation 定义。
- **doc/:** **文档**。包含本指南的 LaTeX 源码及相关资源。

2.3 显式依赖与可复现性

Flakes 机制保证了所有外部依赖的版本都被精确记录在 `flake.lock` 文件中。结合 Nix 的纯函数构建模式, 确保了无论何时何地, 只要使用相同的 Git commit 和 lock 文件, 就能构建出完全相同的系统环境, 实现了高度的可复现性。

3 系统配置管理与部署

本节介绍如何利用此配置管理不同平台的系统, 并指导完成首次环境设置。

3.1 跨平台支持

`flake.nix` 的 `outputs` 通过调用 `lib/mksystem.nix` (或其他类似机制) 生成针对不同平台的配置:

- `nixosConfigurations`: 输出 NixOS 系统配置, 适用于物理机、虚拟机或 WSL。
- `darwinConfigurations`: 输出 nix-darwin 配置, 适用于 macOS 系统。

这种结构使得可以为不同平台 (如 `x86_64-linux`, `aarch64_linux`, `aarch64_darwin`) 定义和构建系统。

3.2 主机特化 (`machines/`)

如架构部分所述, `machines/` 目录用于存放特定主机的配置, 允许根据硬件或运行环境进行定制, 同时复用 `modules/` 中的通用逻辑。

3.3 首次环境部署实践

以下是在不同目标平台上部署此 Nix 配置的详细步骤。

3.3.1 部署到 NixOS 虚拟机 (以 VMware Fusion 为例)

这是创建标准化开发环境的常用方法。

1. 准备 ISO: 从 NixOS 官网下载对应架构 (x86_64 或 aarch64) 的最小化 ISO 镜像。

2. 创建 VM (VMware):

- 使用 ISO 创建新虚拟机。
- **关键设置:** 磁盘接口 SATA (确保证认设备为 `/dev/sda`, 否则需调整 `Makefile` 的 `NIXDEVICE`), 分配足够资源 (CPU, RAM, 建议 $\geq 150\text{GB}$ 磁盘空间), 启用 3D 加速 (如果需要 GUI), 网络模式选择”与 Mac 共享”或桥接模式, 移除不必要的虚拟设备 (声卡等), 启动模式设为 UEFI。
- **VMware 提示:** 在 VMware 设置中禁用 Mac 主机快捷键, 避免与 VM 内快捷键冲突。

3. 启动 VM 并设置 root 密码: 从 ISO 启动进入 NixOS 安装环境, 打开终端:

```
1 sudo su
2 passwd
3 # 输入一个临时的 root 密码, 例如 "root" (此密码仅用于 bootstrap 阶段)
```

4. (可选) 创建快照: 在 VMware 中为”干净”的安装环境创建一个快照 (如”prebootstrap0”), 方便出错时恢复。

5. 获取 VM IP 地址: 在 VM 的终端运行 `ip addr` (或 `ifconfig`), 记下其 IP 地址 (通常是 `192.168.x.y` 或 `172.16.x.y` 等本地地址)。

6. 配置 Makefile (在你的主机上):

- 克隆本 Nix 配置仓库到你的主机 (Mac 或 Linux)。
- 编辑仓库根目录下的 `Makefile`, 找到并修改 `NIXADDR` 变量, 使其指向你刚刚获取的 VM IP 地址。

- **指定目标主机名:** 根据你的 VM 架构和用途, 在主机终端设置环境变量 `NIXNAME`, 使其指向 `flake.nix` 中定义的目标 `nixosConfiguration` 名称。例如, 对于 ARM 架构 VM, 可能运行 `export NIXNAME=vm-aarch64`。对于 Intel VM, 可能是 `export NIXNAME=vm-intel` (默认)。请查阅 `flake.nix` 确认可用的主机名。

7. 执行第一阶段引导 (主机 -> VM): 在主机终端, 切换到 Nix 配置仓库的根目录, 运行:

```
1 make vm/bootstrap0
```

此命令通过 SSH 连接到 VM (使用 root 用户和刚才设置的密码), 自动执行磁盘分区、格式化 (可能使用 `disko`)、生成基础 NixOS 配置 (包含 SSH 服务和 Flake 支持), 然后安装 NixOS 到硬盘。完成后 VM 会自动重启。

8. 执行第二阶段引导 (主机 -> VM): 等待 VM 从硬盘成功启动 (你可以尝试 `ssh root@<NIXADDR>` 来确认 SSH 服务是否可用), 然后在主机终端再次运行:

```
1 make vm/bootstrap
```

此命令会执行以下操作:

- 将你的完整 Nix 配置 (整个仓库) 通过 SSH (`scp`) 复制到 VM 的指定位置 (例如 `/nix-config`)。
- 在 VM 上以 root 身份执行 `nixos-rebuild switch --flake /nix-config#$NIXNAME`, 应用你在 `NIXNAME` 中指定的全套配置。
- (可选, 取决于配置) 可能包含同步主机 SSH/GPG 密钥到 VM 中你定义的用户家目录的逻辑。
- 再次重启 VM 以确保所有服务和设置生效。

部署完成! 现在你可以使用你在用户配置中定义的用户名和密码登录这个完全由 Nix 管理的 VM 了。

3.3.2 部署到 macOS (使用 `nix-darwin`)

1. 安装 Nix: 确保你的 Mac 上已安装 Nix, 并且启用了 Flakes 功能。推荐使用官方安装脚本或 Determinate Systems 的 Nix Installer。

2. 克隆仓库: 将本 Nix 配置仓库克隆到你的 Mac 本地, 例如 `~/code/nix-config`。

3. 选择配置名称: 确认 `flake.nix` 的 `outputs.darwinConfigurations` 中为你这台 Mac 定义的配置名称, 例如 `macbook-pro-m1`。

4. 应用配置: 在你的 Mac 终端中, 切换到 Nix 配置仓库的根目录, 然后运行:

```
1 # 如果你的 Mac 配置名称不是默认值, 先设置环境变量
2 # export NIXNAME=macbook-pro-m1
3 make switch
```

Makefile 检测到 Darwin 环境后, 会自动调用 `darwin-rebuild switch --flake .#$NIXNAME`。首次执行会下载和构建大量依赖, 耗时较长。

重要提示: 在首次应用配置前, 请务必仔细阅读 `flake.nix` 以及与你 NIXNAME 相关的 `machines/<name>.nix`, `users/<user>/darwin.nix`, 和 `users/<user>/home-manager.nix` 文件, 确保其中的设置符合你的预期, 避免覆盖重要系统配置或个人数据。

3.3.3 构建并部署到 WSL

1. 构建 WSL Tarball (在 Linux/macOS 构建环境): 首先确保 `flake.nix` 的 `outputs.nixosConfigurations` 中定义了一个专门为 WSL 设计的配置 (例如, 名为 `wsl`)。然后, 在你的 Nix 构建环境 (可以是已配置好的 NixOS VM 或 Mac) 中, 切换到仓库根目录, 运行:

```
1 make wsl
2 # 或者直接 nix build .#nixosConfigurations.wsl.config.system.build.
   ↳ tarball
```

这会构建出 WSL 发行版所需的 rootfs tarball, 通常位于 `./result/tarball/nixos-wsl.tar.gz` (具体路径取决于 `make wsl` 的实现)。

2. 导入到 Windows WSL: 将生成的 `.tar.gz` 文件复制到你的 Windows 机器上。打开 PowerShell 或 CMD, 执行以下命令:

```
1 # 1. 为你的 NixOS 发行版创建一个安装目录
2 md C:\textbackslash wsl\textbackslash nixos
3
4 # 2. 导入 tarball (替换路径为实际路径)
5 wsl --import nixos C:\textbackslash wsl\textbackslash nixos C:\
   ↳ textbackslash path\textbackslash to\textbackslash your\
   ↳ textbackslash nixos-wsl.tar.gz --version 2
6
```



```
7 # 3. 启动导入的 NixOS 发行版
8 wsl -d nixos
9
10 # 4. (可选) 将 NixOS 设置为默认 WSL 发行版
11 wsl --set-default nixos
```

之后，你就可以通过 Windows Terminal 或运行 `wsl -d nixos` 来进入你的 NixOS WSL 环境了。

4 用户环境管理 (Home Manager)

Home Manager 是管理用户级别配置的核心工具，允许你声明式地定义 dotfiles、软件包、服务和环境变量等。

4.1 用户配置结构 (users/)

如架构部分所述，`users/` 目录下的每个子目录代表一个用户。其中的 `home-manager.nix` 文件是该用户环境配置的主要入口点。

4.2 常见任务：管理软件包

管理用户独立安装的软件包是 Home Manager 的常用功能。

- **为当前用户添加包：**编辑你自己的 `users/< 你的用户名 >/home-manager.nix` 文件，找到或添加 `home.packages = [ ... ]`；列表，然后在列表中加入你想安装的包名，使用 `pkgs.<packageName>` 的形式，例如 `pkgs.htop`, `pkgs.ripgrep`。
- **为当前用户删除包：**从上述 `home.packages` 列表中移除对应的包名即可。
- **应用更改：**修改完 `home-manager.nix` 后，在仓库根目录运行 `make switch` 来应用更改。Home Manager 会自动安装或卸载指定的包。

注意：对于需要在系统层面全局可用的包，应在对应的 `machines/< 机器名 >.nix` (或 `users/< 用户 >/nixos.nix`) 的 `environment.systemPackages` 中管理。

4.3 常见任务：修改用户环境

Home Manager 提供了丰富的选项来定制用户环境。

- **Shell 配置:** 在 `home-manager.nix` 中查找类似 `programs.zsh = { enable = true; ... };` 或 `programs.fish = { enable = true; ... };` 的配置块, 修改其中的选项来自定义你的 Shell (例如, 启用插件、设置别名、修改提示符)。
- **环境变量:** 使用 `home.sessionVariables = { VAR_NAME = "value"; };` 来设置用户会话的环境变量。
- **Git 配置:** 通过 `programs.git = { enable = true; userName = "..."; userEmail = "..."; ... };` 配置 Git。
- **应用程序配置:** Home Manager 支持许多应用程序的声明式配置, 例如 Neovim (`programs.neovim`), Tmux (`programs.tmux`) 等。查阅 Home Manager 文档以了解特定程序的支持情况。
- **应用更改:** 同样, 修改完后运行 `make switch` 应用。

5 高级功能与定制

本节介绍一些更高级的配置定制方法。

5.1 常见任务: 使用 Overlays 修改或添加包

当你需要修改 Nixpkgs 中的某个包 (例如, 应用一个未被上游接受的补丁) 或者添加一个简单的自定义包时, 可以使用 Overlays。

1. 创建 Overlay 文件: 在 `overlays/` 目录下创建一个 `.nix` 文件, 例如 `my-patches.nix`。

2. 定义 Overlay 函数: 该文件应返回一个函数, 其参数通常为 `self` 和 `super`。`self` 代表最终的 Nixpkgs 包集 (包含所有 Overlays), `super` 代表应用当前 Overlay 之前的包集。例如:

```

1 # overlays/my-patches.nix
2 self: super: {
3   # 示例 1: 修改现有包 htop, 比如应用一个本地补丁
4   htop = super.htop.overrideAttrs (oldAttrs: {
5     patches = (oldAttrs.patches or []) ++ [ ./my-htop.patch ];
6     # 你也可以修改 version, src, buildInputs, buildPhase 等
7   });
8
9   # 示例 2: 添加一个简单的包 (假设定义在同目录的 my-tool.nix)
```

```

10   # my-tool = super.callPackage ./my-tool.nix {};
11
12   # 示例 3: 添加一个引用自定义 pkgs/ 目录下包的别名
13   # my-complex-tool = self.callPackage ../pkgs/my-complex-tool {};
14 }

```

确保你的补丁文件 (如 my-htop.patch) 也放在仓库中, 路径相对于 Overlay 文件。

3. 应用 Overlay: 通常, lib/mkssystem.nix 会被设计为自动加载 overlays/ 目录下的所有 .nix 文件, 并将它们应用到传递给 Home Manager 和系统配置的 pkgs 实例上。如果不是自动加载, 你可能需要在 flake.nix 中显式导入并传递它们。请检查你的 lib/mkssystem.nix 实现。

4. 构建系统: 运行 make switch。现在系统将使用你通过 Overlay 修改或添加的包。

对于更复杂的自定义包, 建议放在 pkgs/ 目录下独立管理。

5.2 常见任务: 添加新的系统配置

当你需要为一台新的物理机、虚拟机或特定用途 (如一个新的 WSL 配置) 添加配置时:

1. 创建机器配置文件: 在 machines/ 目录下, 为新系统创建一个 Nix 文件, 例如 machines/new-server.nix 定义该机器特有的硬件设置、内核模块、文件系统配置或系统服务。可以参考现有的机器配置作为模板。

2. 在 flake.nix 中添加输出: 编辑 flake.nix 文件, 在 outputs 函数内部, 模仿现有的 nixosConfigurations 或 darwinConfigurations 条目, 添加一个新的条目。这通常涉及到调用 lib/mkssystem.nix 函数, 并传入新机器的名称和相关参数:

```

1  outputs = { self, nixpkgs, home-manager, ... }@inputs: let
2    # 假设 mkSystem 在这里定义或导入
3    mkSystem = import ./lib/mkssystem.nix { inherit inputs; };
4  in {
5    # ... 其他输出 ...
6
7    nixosConfigurations."new-server" = mkSystem "new-server" {
8      system = "x86_64-linux"; # 或 aarch64-linux, aarch64-darwin 等
9      user = "admin-user";     # 指定默认或主要用户的配置
10     # extraModules = [ ... ]; # 可能需要传入额外的系统模块

```

```

11     # isWSL = true;           # 如果是为 WSL 设计的配置
12 };
13
14 # 如果是 macOS:
15 # darwinConfigurations."new-mac" = mkSystem "new-mac" { ... };
16 };

```

3. 确保用户配置存在: 确认你在上面 `mkSystem` 调用中指定的 `user` (例如 `admin-user`) 在 `users/` 目录下有相应的配置(至少需要 `nixos.nix` 或 `darwin.nix` 以及 `home-manager.nix` 文件)。

4. 在新机器上部署: 将配置仓库克隆到新机器上, 然后根据新机器的平台和配置名称 (`new-server` 或 `new-mac`), 执行相应的首次部署步骤 (参考第 3.3 节) 或直接运行 `make switch` (如果 Nix 已安装且网络可达)。对于需要引导的情况, 可能需要先设置 `export NIXNAME=new-server`。

6 日常维护与 workflow

本节汇总了日常使用和维护此 Nix 配置时最常用的命令。

6.1 应用配置更改

当你修改了任何 `.nix` 配置文件 (无论是系统级、用户级、模块、Overlay 还是包定义) 后, 需要运行以下命令来构建并应用这些更改:

```

1 # 在配置仓库的根目录下
2 # 对于非默认主机/用户配置, 可能需要先设置 NIXNAME
3 # export NIXNAME=my-other-machine
4 make switch

```

这个命令会根据当前环境 (NixOS 或 Darwin) 调用相应的 `nixos-rebuild switch` 或 `darwin-rebuild switch`, 并自动传入 Flake 路径和配置名称。

6.2 测试配置更改

在正式应用更改之前, 特别是对于复杂的修改, 建议先测试构建是否成功:

```

1 # 在配置仓库的根目录下
2 # 对于非默认主机/用户配置, 可能需要先设置 NIXNAME

```

```
3 # export NIXNAME=...
4 make test
```

这通常会调用 `nixos-rebuild build` 或 `darwin-rebuild build` (或 `nix build`), 只构建配置而不激活它。如果构建失败, 会报告错误, 避免破坏当前系统。

6.3 更新 Flake 依赖

要将 `flake.nix` 中定义的外部依赖 (如 `nixpkgs`, `home-manager` 等) 更新到其 Flake URL 指向的最新版本, 运行:

```
1 nix flake update
```

这会修改 `flake.lock` 文件以反映新的依赖版本。更新 `lock` 文件后, 你需要运行 `make switch` 来下载、构建并应用这些更新后的依赖及其带来的变化。

6.4 编译本文档

如果你修改了本文档 (`doc/configuration_guide.tex`), 并且你的环境中安装了必要的 LaTeX 工具链 (特别是 XeLaTeX), 可以运行:

```
1 make doc
```

这通常会调用 `xelatex` 命令在 `doc/` 目录下生成 PDF 文件 (`configuration_guide.pdf`)。

安装 LaTeX: 如果缺少 LaTeX, 可以通过将 `pkgs.texlive.combined.scheme-full` (或更小的 `scheme`) 添加到你的 `home.packages` (或系统包) 并运行 `make switch` 来安装。

7 结论

该 Nix 配置仓库通过精心设计的模块化结构和对 Nix Flakes 的运用, 提供了一个强大、灵活且可复现的跨平台系统与用户环境管理解决方案。

- 其清晰的关注点分离和代码复用实践, 显著提高了复杂配置的可维护性和可扩展性。
- Flakes 带来的显式依赖管理和构建确定性, 确保了环境的一致性和可靠性。
- 通过整合 Home Manager、Overlays 等工具, 实现了从底层系统到用户应用程序的全面声明式管理。

- 提供的 **Makefile** 和详细的部署指南简化了上手和日常操作流程。

对于追求高效、稳定和一致性配置管理的个人开发者或团队而言，这套配置及其背后的设计原则提供了一个优秀的实践范例和坚实基础。